

Static Code Analysis

Lecture 7

Stanislav Chren

stanislav.chren@aalto.fi

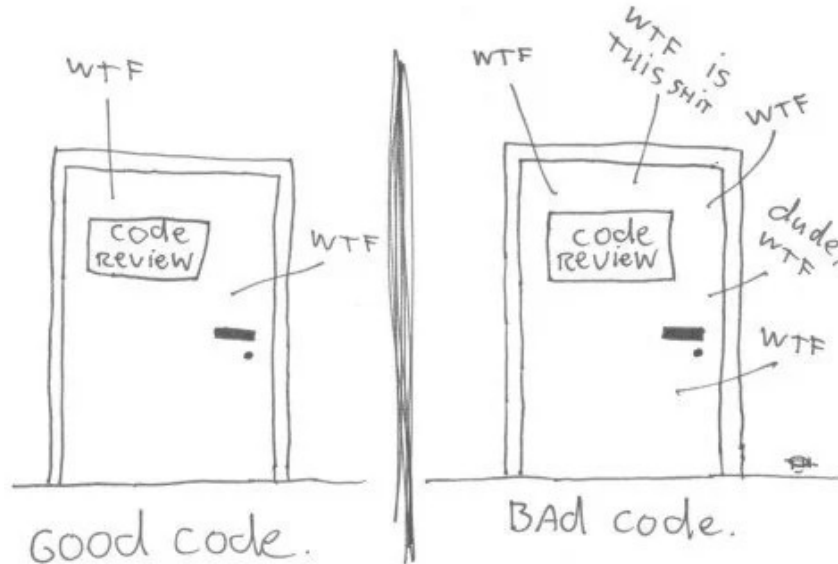


Aalto University
School of Science



Code Quality

The ONLY valid measurement
of code quality: WTFs/minute

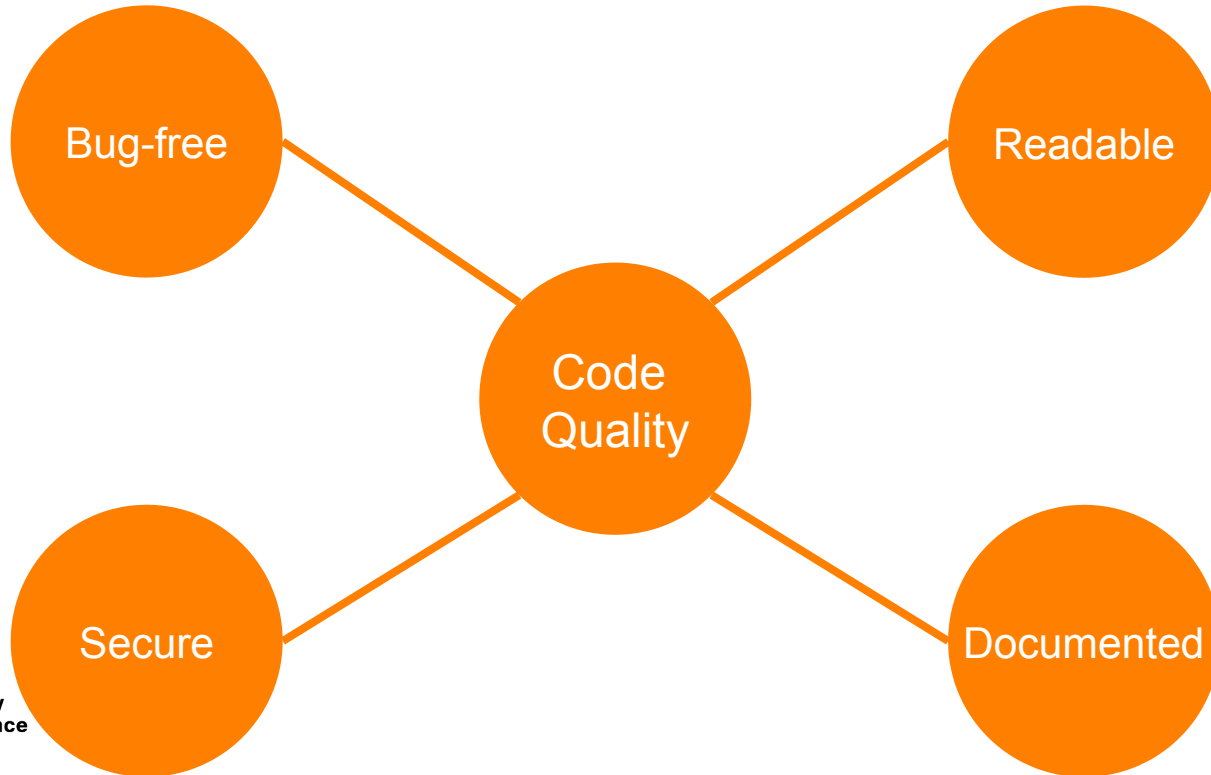


Code Quality

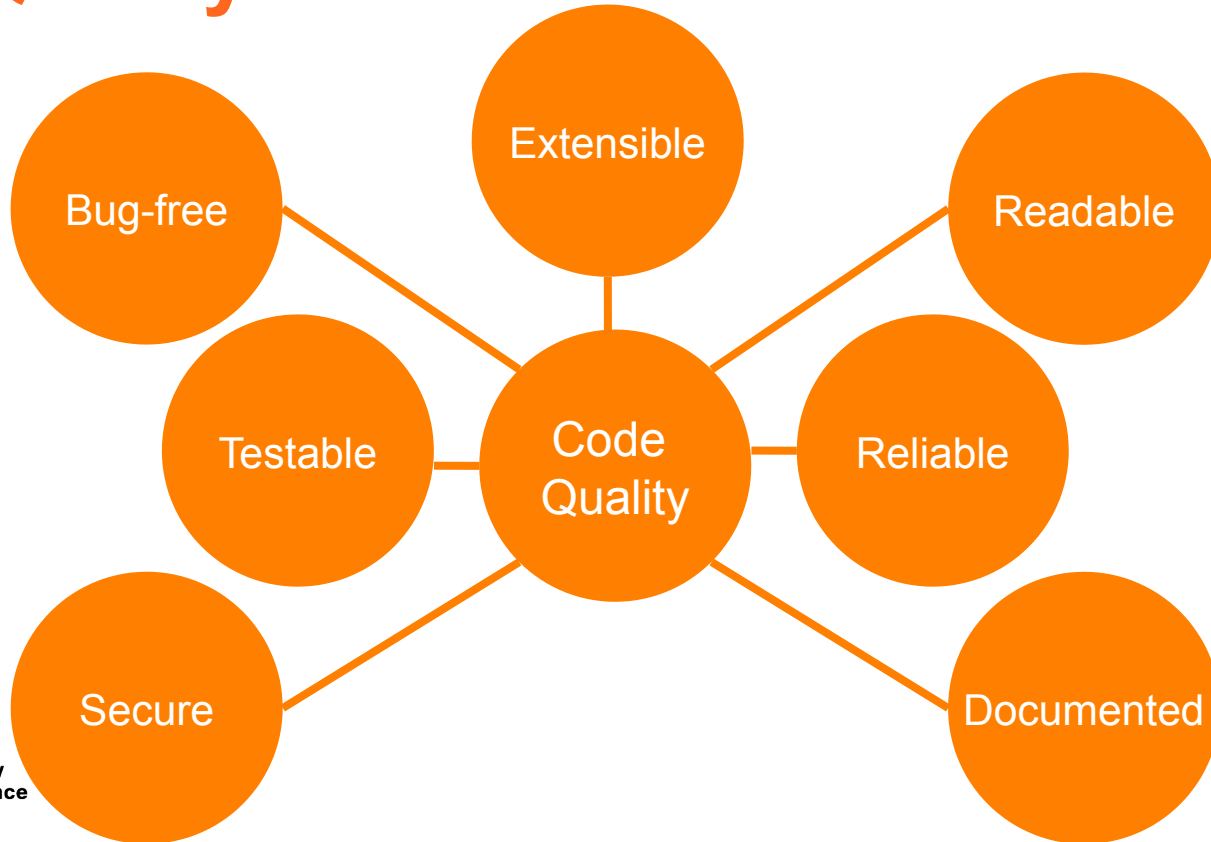


Code
Quality

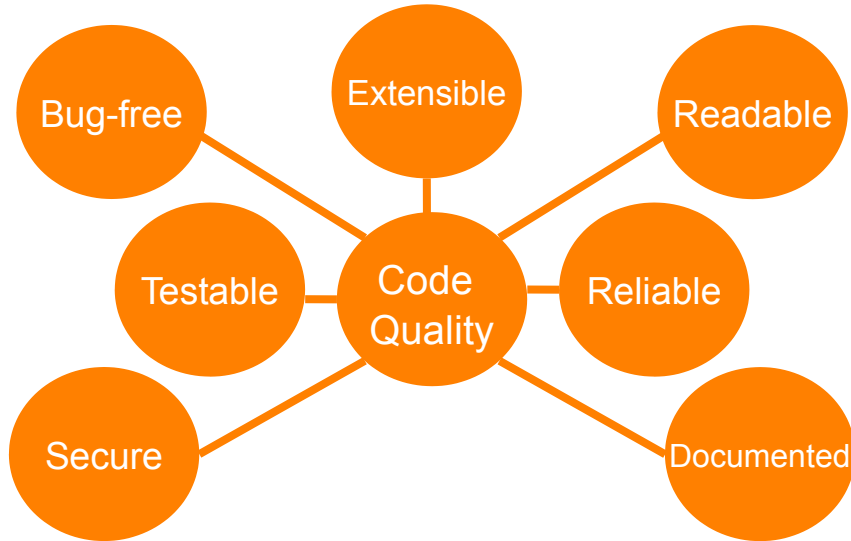
Code Quality



Code Quality



Code Quality



- No established code quality standards
- Guideliness or recommendeations
 - ~ SOLID
 - ~ Clean Code
 - ~ DRY
 - ~ KISS
- Evaluated manually (code reivew) or automatically (SCA, ...)

Code Quality - SOLID

- **S**ingle responsibility principle
- **O**pen-closed principle
- **L**iskov substitution principle
- **I**nterface segregation principle
- **D**ependency inversion principle

Code Quality - SOLID

- Single responsibility principle

```
public class Invoice {  
    ...  
    public Invoice(Book book, int quantity, double discountRate, double taxRate) {  
        ....  
    }  
    public double calculateTotal() {  
        return (book.price - book.price * discountRate) * this.quantity;  
    }  
    public void printInvoice() {  
        System.out.println(quantity + "x " + book.name + " " + book.price + "$");  
        System.out.println("Discount Rate: " + discountRate);  
        ....  
    }  
    public void saveToFile(String filename) {  
        // Creates a file with given name and writes the invoice  
    }  
}
```

Code Quality - SOLID

- Single responsibility principle

```
public class Invoice {  
    ...  
    public Invoice(Book book, int quantity, double discountRate, double taxRate) {  
        ....  
    }  
    public double calculateTotal() {  
        return (book.price - book.price * discountRate) * this.quantity;  
    }  
    public void printInvoice() {  
        System.out.println(quantity + "x " + book.name + " " + book.price + "$");  
        System.out.println("Discount Rate: " + discountRate);  
        ....  
    }  
    public void saveToFile(String filename) {  
        // Creates a file with given name and writes the invoice  
    }  
}
```

Code Quality - SOLID

- Single responsibility principle
- Open-closed principle

```
public class InvoicePersistence {  
    Invoice invoice;  
    public InvoicePersistence(Invoice invoice) {  
        this.invoice = invoice;  
    }  
    public void saveToFile(String filename) {  
        // Creates a file with given name and writes the invoice  
    }  
  
}
```

Code Quality - SOLID

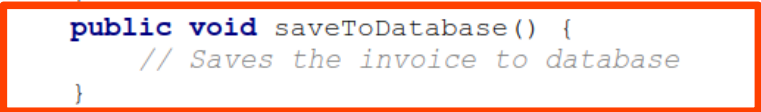
- Single responsibility principle
- Open-closed principle

```
public class InvoicePersistence {  
    Invoice invoice;  
    public InvoicePersistence(Invoice invoice) {  
        this.invoice = invoice;  
    }  
    public void saveToFile(String filename) {  
        // Creates a file with given name and writes the invoice  
    }  
    public void saveToDatabase() {  
        // Saves the invoice to database  
    }  
}
```

Code Quality - SOLID

- Single responsibility principle
- Open-closed principle

```
public class InvoicePersistence {  
    Invoice invoice;  
    public InvoicePersistence(Invoice invoice) {  
        this.invoice = invoice;  
    }  
    public void saveToFile(String filename) {  
        // Creates a file with given name and  
    }  
    public void saveToDatabase() {  
        // Saves the invoice to database  
    }  
}
```



```
interface InvoicePersistence {  
    public void save(Invoice invoice);  
}  
  
public class DatabasePersistence implements InvoicePersistence {  
    @Override  
    public void save(Invoice invoice) {  
        // Save to DB  
    }  
}  
  
public class FilePersistence implements InvoicePersistence {  
    @Override  
    public void save(Invoice invoice) {  
        // Save to file  
    }  
}
```

Code Quality - SOLID

- Single responsibility principle
- Open-closed principle
- Liskov substitution principle

```
public class Bird{
    public void fly();
    public void walk();
    ...
}
public class Parrot extends Bird {
    public void fly(){ ...}
    public void walk(){ ... }
}
public class Penguin extends Bird {
    public void fly(){ ... }
    public void walk(){ ... }
```

A?

Code Quality - SOLID

- Single responsibility principle
- Open-closed principle
- Liskov substitution principle

```
public class Bird{  
    public void fly();  
    public void walk();  
    ...  
}  
  
public class Parrot extends Bird {  
    public void fly(){ ...}  
    public void walk(){ ... }  
}  
  
public class Penguin extends Bird {  
    public void fly(){ ... }  
    public void walk(){ ... }  
}
```



```
public class Bird{  
    public void walk();  
    ...  
}  
  
public class Parrot extends Bird {  
    public void fly(){ ...}  
    public void walk(){ ... }  
}  
  
public class Penguin extends Bird {  
    public void walk(){ ... }  
}
```

A?

Code Quality - SOLID

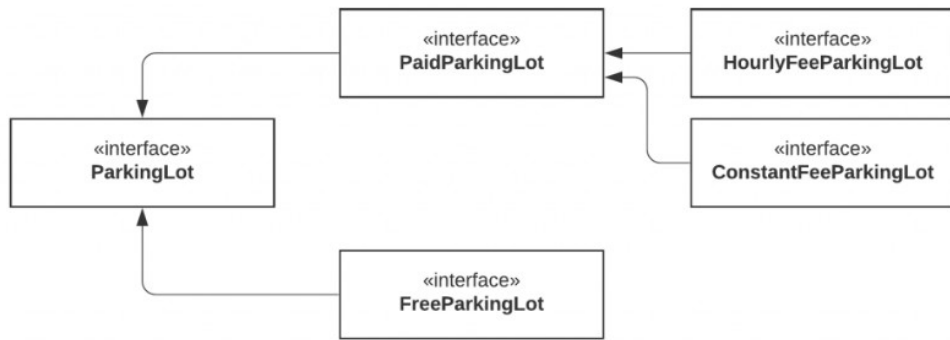
- Single responsibility principle
- Open-closed principle
- Liskov substitution principle
- Interface segregation principle

```
public interface ParkingLot {  
    void parkCar();  
    void unparkCar();  
    void getCapacity();  
    double calculateFee(Car car);  
    void doPayment(Car car);  
}
```

Code Quality - SOLID

- Single responsibility principle
- Open-closed principle
- Liskov substitution principle
- Interface segregation principle

```
public interface ParkingLot {  
    void parkCar();  
    void unparkCar();  
    void getCapacity();  
    double calculateFee(Car car);  
    void doPayment(Car car);  
}
```



Code Quality - SOLID

- Single responsibility principle
- Open-closed principle
- Liskov substitution principle
- Interface segregation principle
- Dependency inversion principle

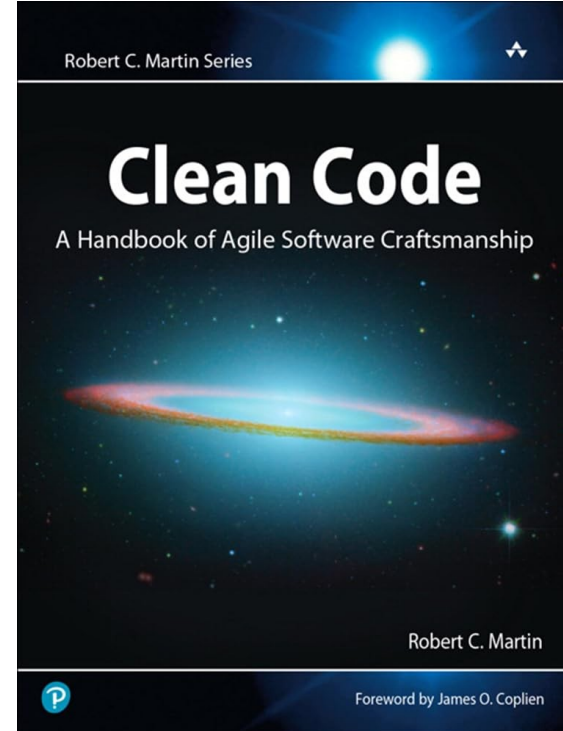
```
class MySQLDatabase {
    public void get() {...}
    public void insert() {...}
    public void update() {...}
    public void delete() {...}
}

class BudgetReport {
    private MySQLDatabase database;
    public BudgetReport(MySQLDatabase database) {
        this.database = database;
    }
    public void open() {
        database.get();
    }
    public void save() {
        database.insert();
    }
}

// Client code
public class Main {
    public static void main(String[] args) {
        MySQLDatabase database = new MySQLDatabase();
        BudgetReport report = new BudgetReport(database);
        report.open();
    }
}
```

Code Quality – Clean Code

- Collection of **recommendations**
 - Naming
 - Code structure
 - Error handling
 - Documentation
 - Unit testing
 -



Code Quality – Clean Code

```
public int[] generatePrimes(int maxValue) {
    if (maxValue >= 2) // the only valid case {
        // declarations
        int s = maxValue + 1; // size of array
        boolean[] f = new boolean[s];
        int i;
        // initialize array to true.
        for (i = 0; i < s; i++)
            f[i] = true;
        // get rid of known non-primes
        f[0] = f[1] = false;
        // sieve
        int j;
        for (i = 2; i < Math.sqrt(s) + 1; i++) {
            if (f[i]) { // if i is uncrossed, cross its multiples.
                for (j = 2 * i; j < s; j += i)
                    f[j] = false; // multiple is not prime
            }
        }
        // how many primes are there?
        int count = 0;
        for (i = 0; i < s; i++) {
            if (f[i])
                count++; // bump count.
        }
        int[] primes = new int[count];
        // move the primes into the result
        for (i = 0, j = 0; i < s; i++) {
            if (f[i]) // if prime
                primes[j++] = i;
        }
        return primes; // return the primes
    }
    else // maxValue < 2
        return new int[0]; // return null array if bad input.
}
```



```
public int[] generatePrimes(int maxValue) {
    if (maxValue < 2)
        return new int[0];
    else {
        generateUncrossedIntegersUpTo(maxValue);
        crossOutMultiples();
        putUncrossedIntegersIntoResult();
        return result;
    }
}

private void generateUncrossedIntegersUpTo(int maxValue) {
    crossedOut = new boolean[maxValue + 1];
    for (int i = 2; i < crossedOut.length; i++)
        crossedOut[i] = false;
}

private void crossOutMultiples() {
    int limit = determineIterationLimit();
    for (int i = 2; i <= limit; i++)
        if (notCrossed(i))
            crossOutMultiplesOf(i);
}

private int determineIterationLimit() {
    double iterationLimit = Math.sqrt(crossedOut.length);
    return (int) iterationLimit;
}

private void crossOutMultiplesOf(int i) {
    for (int multiple = 2 * i;
         multiple < crossedOut.length;
         multiple += i)
        crossedOut[multiple] = true;
}

private boolean notCrossed(int i) {
    return crossedOut[i] == false;
}

private void putUncrossedIntegersIntoResult() {
    result = new int[numberOfUncrossedIntegers()];
    for (int j = 0, i = 2; i < crossedOut.length; i++)
        if (notCrossed(i))
            result[j++] = i;
}

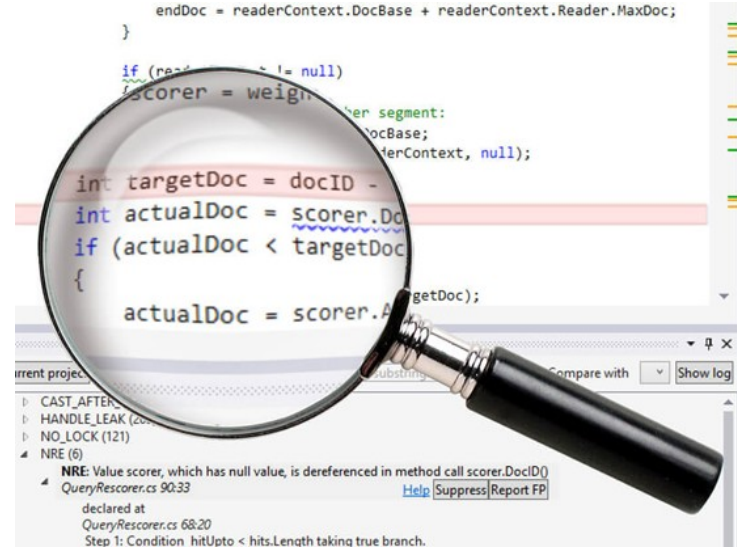
private int numberOfUncrossedIntegers() {
    int count = 0;
    for (int i = 2; i < crossedOut.length; i++)
        if (notCrossed(i))
            count++;
    return count;
}
```

What is Static Code Analysis?

- **Analysis of computer software that is performed without actually executing programs.**

~ Performed on a version of the source code, or some form of the object code.

~ The term is usually applied to the analysis performed by an automated tool.



Source: <https://was-hillt.info/references/static-vs-dynamic-source-code-analysis.php>

Discussion

- What are the possible applications of SCA?
- What are the benefits/drawbacks of SCA?



Applications of SCA

Type checking

- Checks correct assignment of object types

```
def headline(text: str, align: bool = True):  
    if align:  
        return f"{text.title()}\n{'-' * len(text)}"  
    else:  
        return f" {text.title()} ".center(50, "o")  
  
print(headline("python type checking"))  
print(headline("use pycharm", "center"))
```

Expected type 'bool', got 'str' instead [more...](#) (Ctrl+F1)

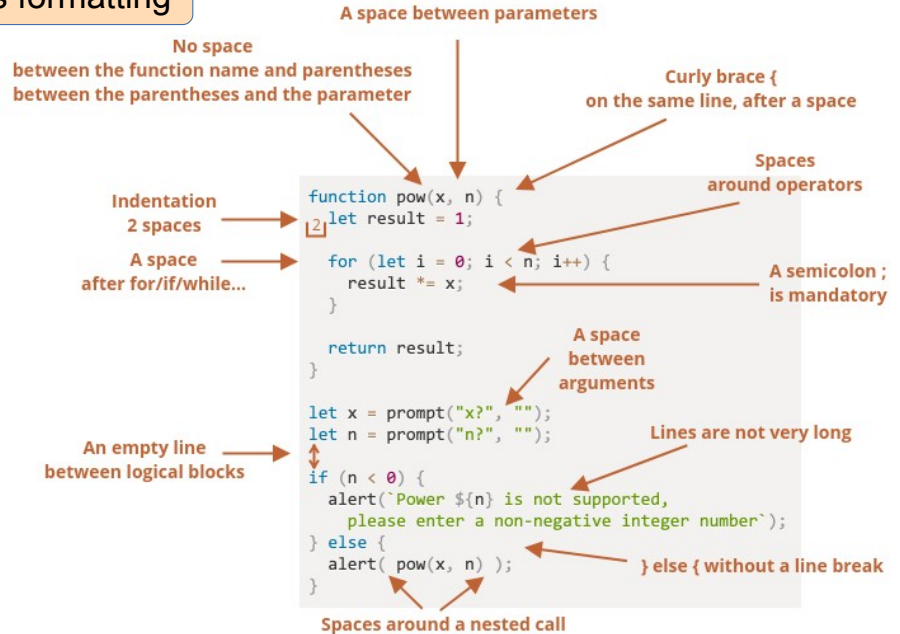
Applications of SCA

Type checking

- Checks correct assignment of object types

Style checking

- Checks style of the code and its formatting



Applications of SCA

Type checking

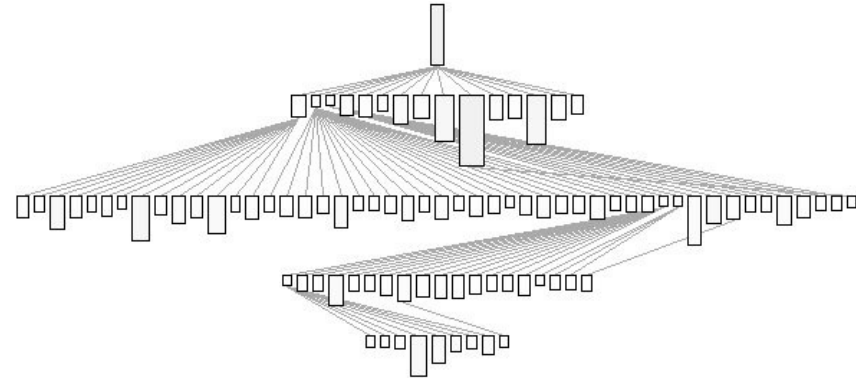
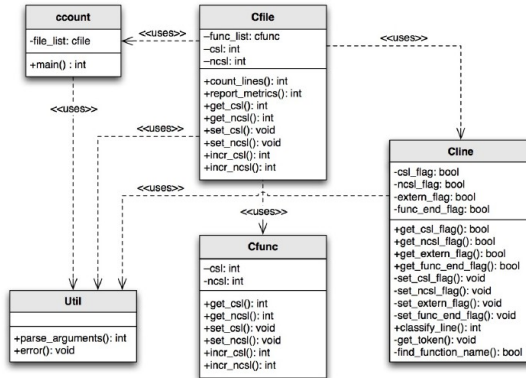
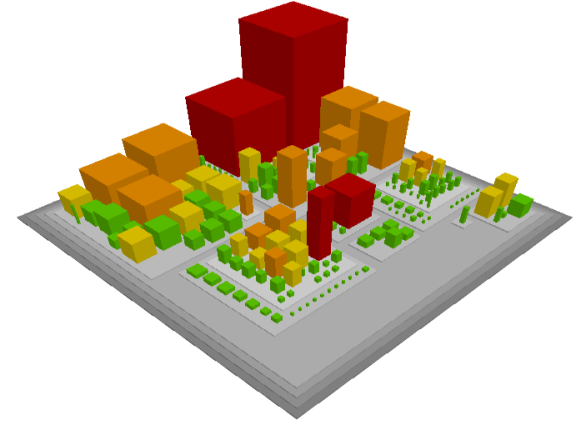
- Checks correct assignment of object types

Style checking

- Checks style of the code and its formatting

Program understanding

- Helps user make sense of large codebase



Applications of SCA

Type checking

- Checks correct assignment of object types

Style checking

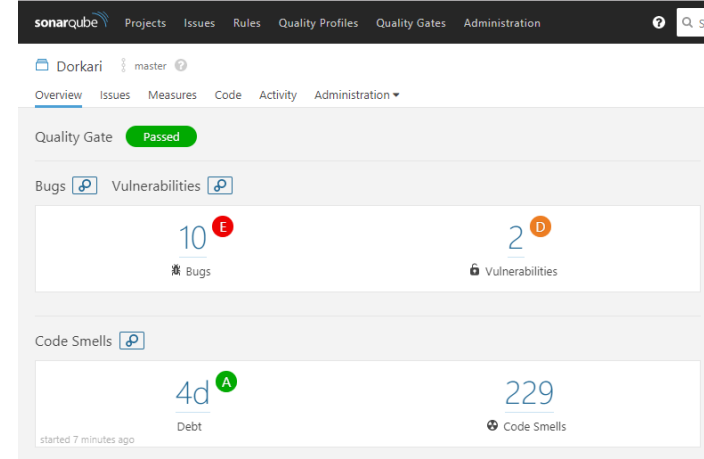
- Checks style of the code and its formatting

Program understanding

- Helps user make sense of large codebase

Issue detection

- Detection of security vulnerabilities
- Detection of bugs
- Detection of code smells



Applications of SCA

Type checking

- Checks correct assignment of object types

Style checking

- Checks style of the code and its formatting

Program understanding

- Helps user make sense of large codebase

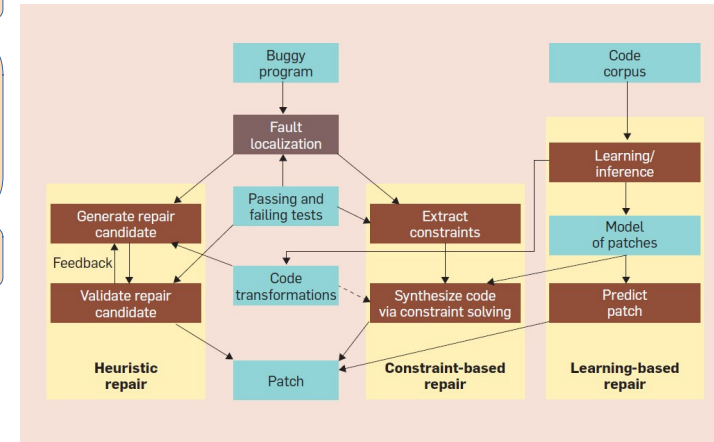
Issue detection

- Detection of security vulnerabilities
- Detection of bugs
- Detection of code smells

Code modification

- Fault injection or code repair

Original Method	With Embedded Mutants
<pre>int Min (int A, int B) { int minVal; minVal = A; if (B < A) { minVal = B; } return (minVal); } // end Min</pre>	<pre>int Min (int A, int B) { int minVal; minVal = A; minVal = B; if (B < A) if (B > A) if (B < minVal) { minVal = B; Bomb(); minVal = A; minVal = failOnZero (B); } return (minVal); } // end Min</pre>



Benefits of SCA

Higher code
quality

Cheaper
bug fixing

No input

Automation

Readability

Coding
guidelines
compliance

Repeatability



Drawbacks of SCA

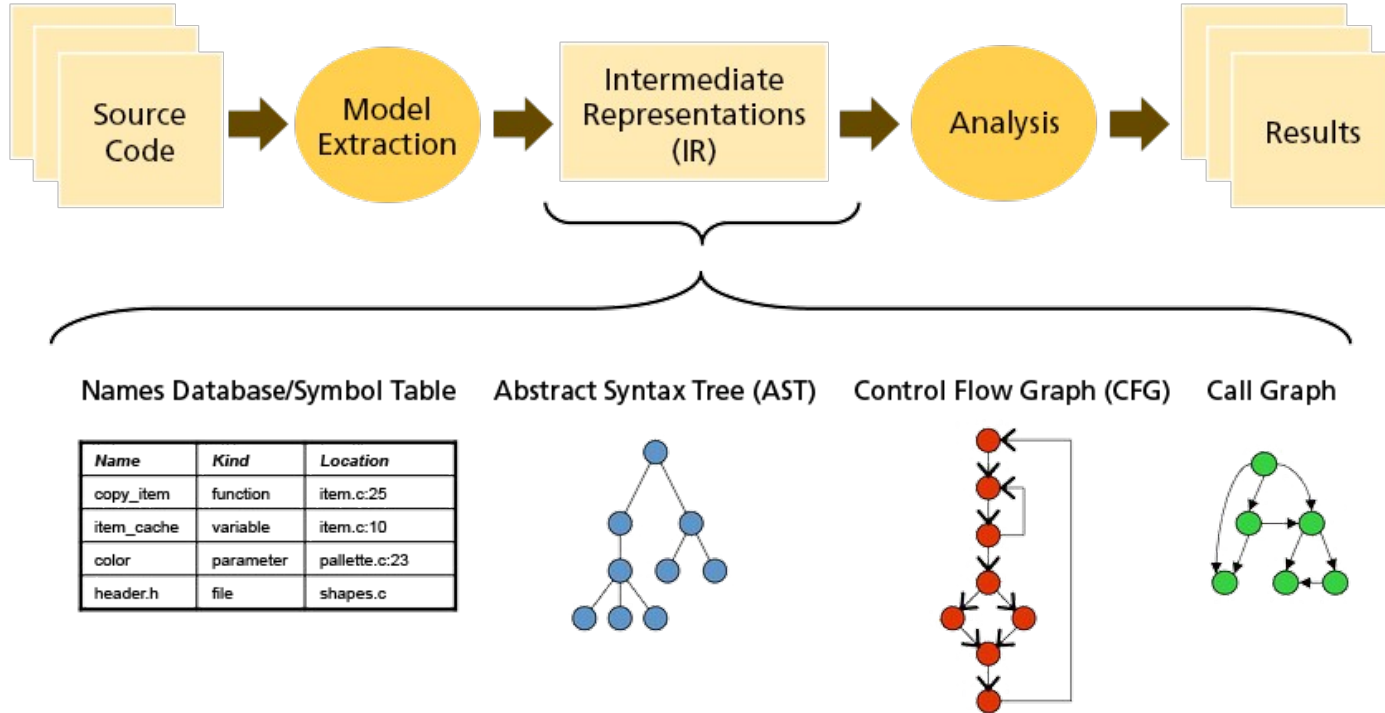
- False positives/negatives
- Only STATIC analysis
- Possible overhead
- Mostly very specialized tools



```
1  public class HelloWorld {  
2  
3      public static void main(String[] args) {  
4          // Prints "Hello, World" to the terminal window.  
5          System.out.println("Hello, World");  
6      }  
7  
8  }
```

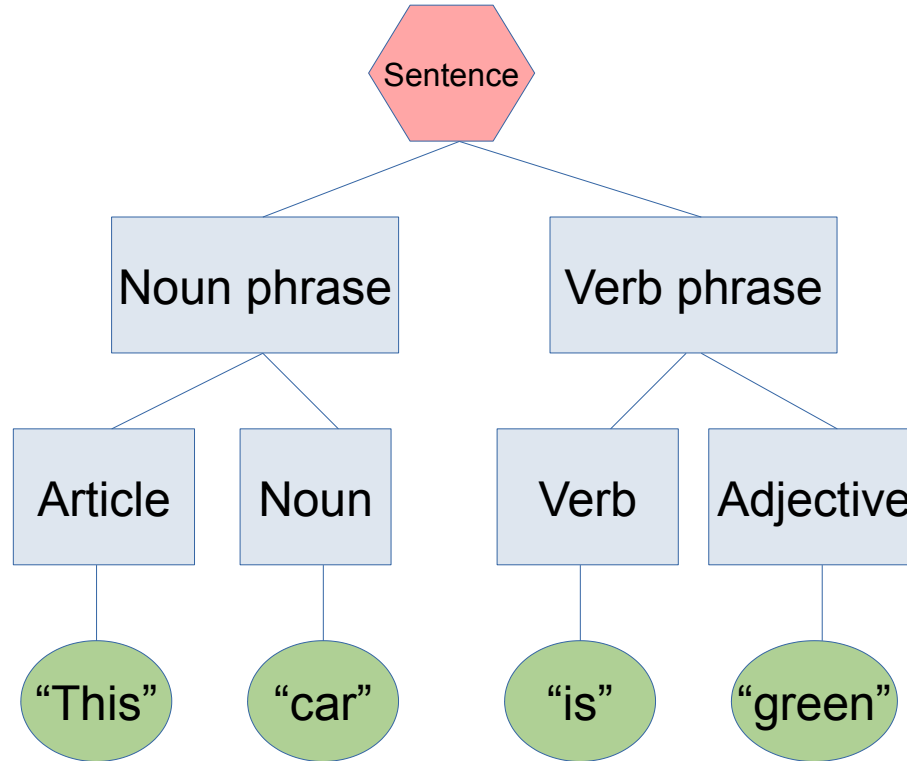
The screenshot displays two static analysis warnings in a code editor. The first warning, located at line 1, suggests adding a private constructor to hide the implicit public one. It is categorized as a 'Code Smell' with a 'Major' severity, marked as 'Open', 'Not assigned', and estimated to take '30min effort'. The second warning, located at line 5, suggests replacing the usage of `System.out` or `System.err` with a logger. It is also categorized as a 'Code Smell' with a 'Major' severity, marked as 'Open', 'Not assigned', and estimated to take '10min effort'. Both warnings are dated '2 months ago' and include a 'design' tag.

How does it work?

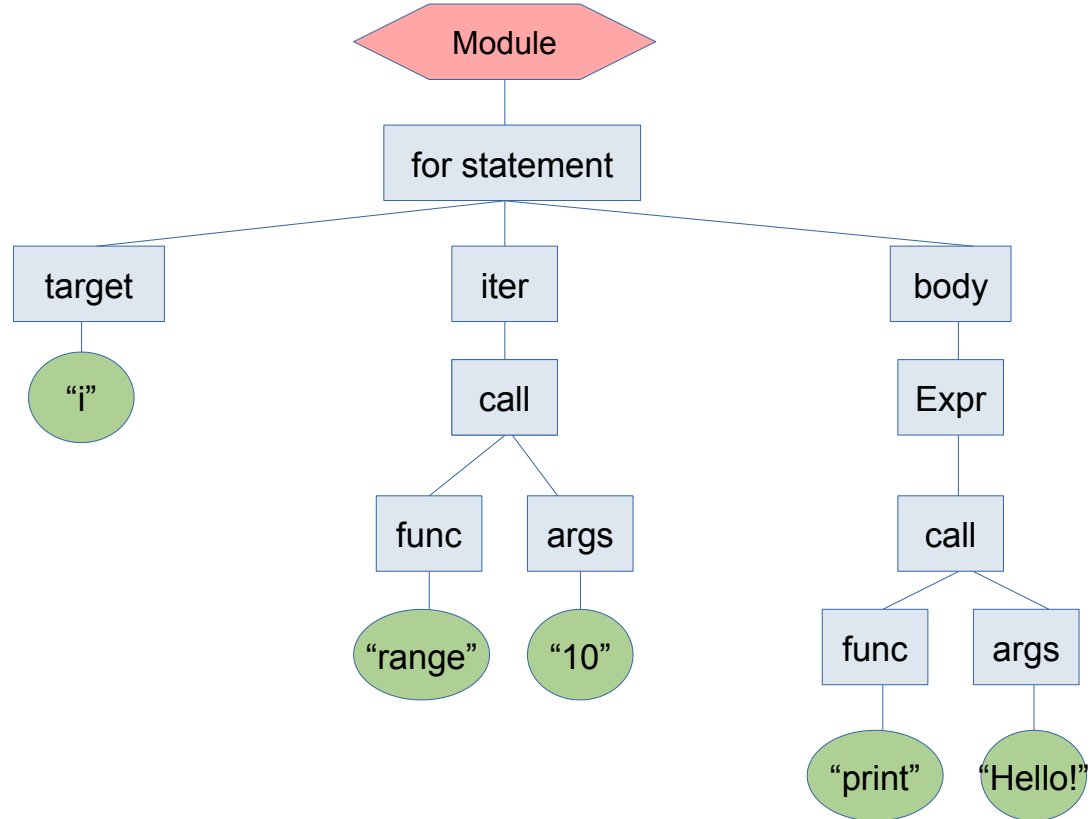


Abstract Syntax Tree (AST)

- This car is green.

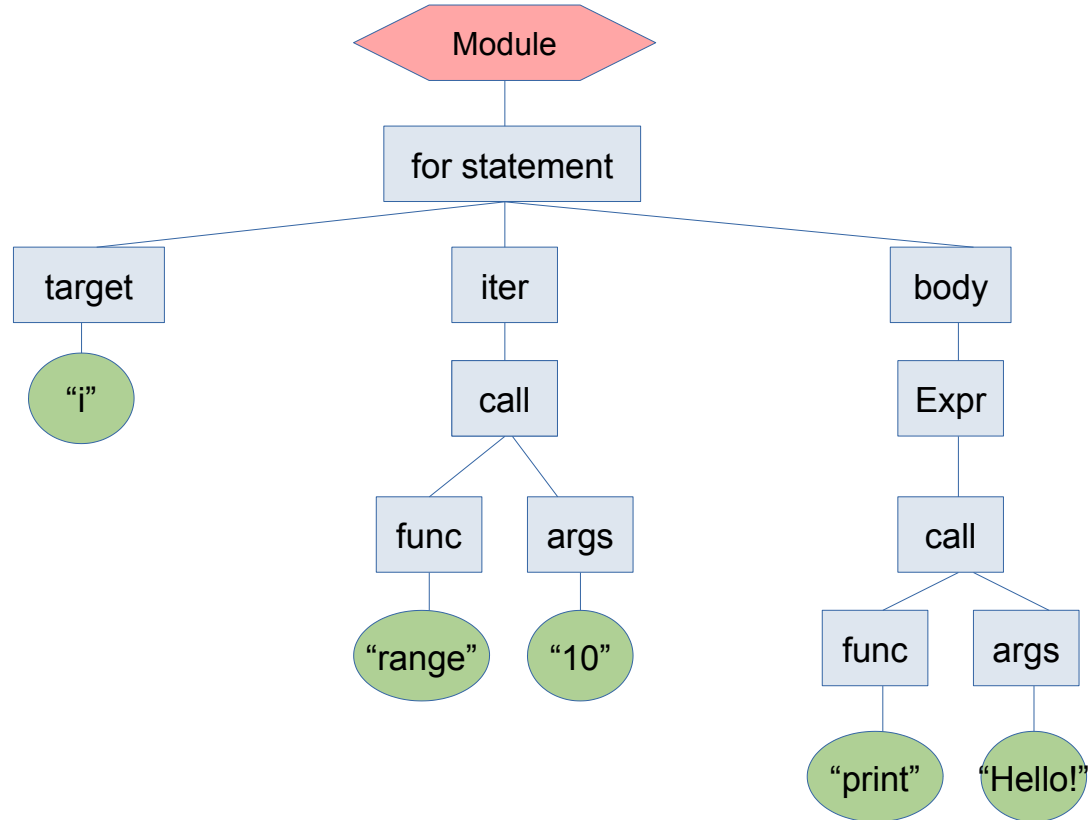


Abstract Syntax Tree (AST)



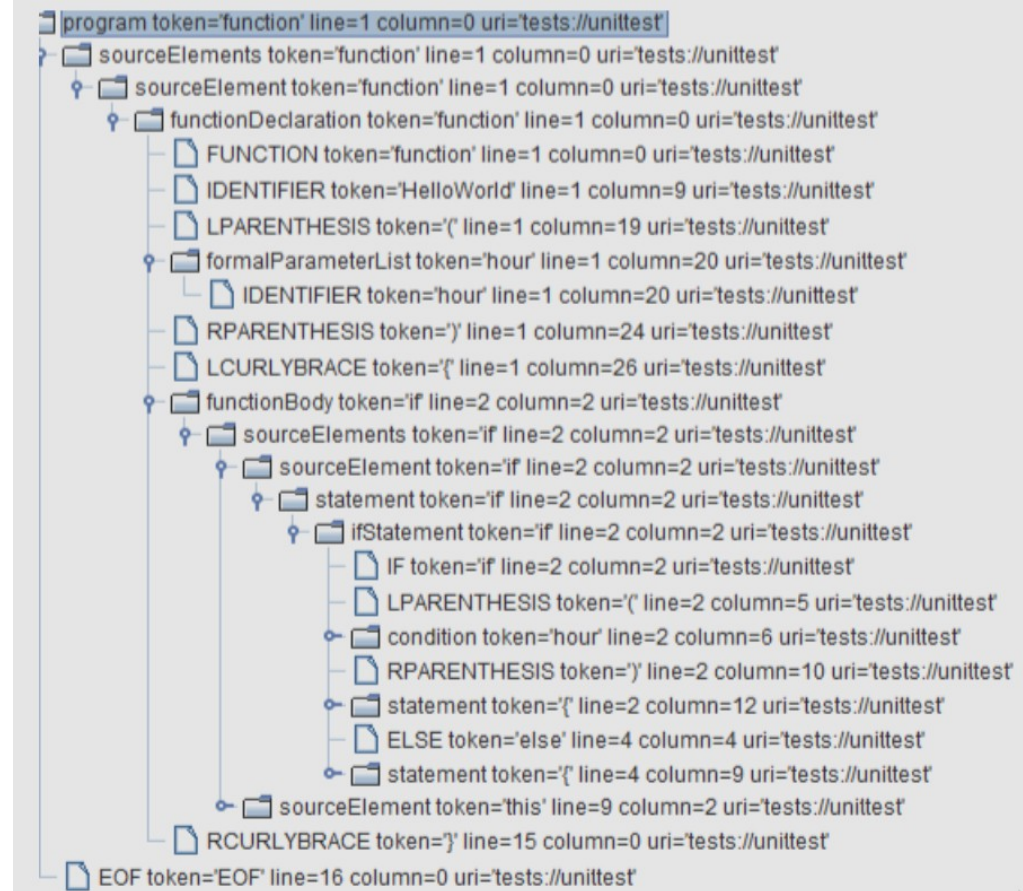
Abstract Syntax Tree (AST)

- for i in range(10):
 print('Hello!')



AST – Example

```
function HelloWorld(hour) {  
  if (hour) {  
    this.hour = hour;  
  } else {  
    var date = new Date();  
    this.hour = date.getHours();  
  }  
  this.displayGreeting = function() {  
    if (this.hour >= 22 || this.hour <= 5)  
      document.write("Good night, World!");  
    else  
      document.write("Hello, World!");  
  }  
}
```

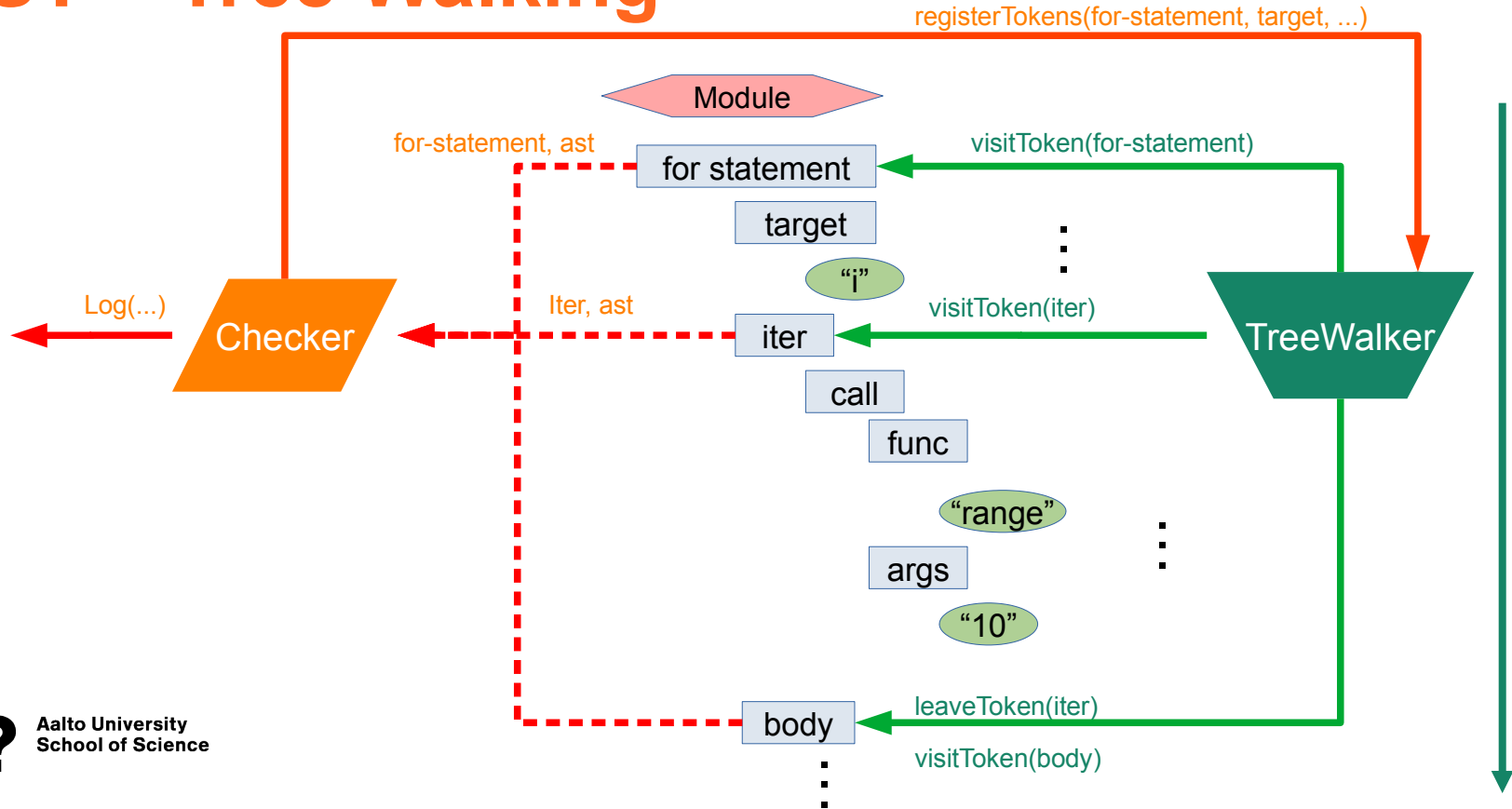


AST – Example

```
function HelloWorld(hour) {  
  if (hour) {  
    this.hour = hour;  
  } else {  
    var date = new Date();  
    this.hour = date.getHours();  
  }  
  this.displayGreeting = function() {  
    if (this.hour >= 22 || this.hour <= 5)  
      document.write("Good night, World!");  
    else  
      document.write("Hello, World!");  
  }  
}
```



AST – Tree Walking



Issue detection - Bug

- Reliability issues
- May crash at runtime
- May cause extremely unpredictable behaviour
- Examples

- ~ Null pointer dereference
- ~ Memory leaks
- ~ Buffer overflow

```
1. class PathManager:
2.     def __init__(self):
3.         self.paths = {}
4.
5.     def add_path(self, name, path):
6.         self.paths[name] = path
7.
8.     def get_normalized_path(self, name):
9.         return self.paths.get(name).lower()
```

Issue detection - Bug

- Reliability issues
- May crash at runtime
- May cause extremely unpredictable behaviour
- Examples

~ Null pointer dereference

~ Memory leaks

~ Buffer overflow

```
1. class PathManager:
2.     def __init__(self):
3.         self.paths = {}
4.
5.     def add_path(self, name, path):
6.         self.paths[name] = path
7.
8.     def get_normalized_path(self, name):
9.         return self.paths.get(name).lower()
```

Issue detection - Vulnerability

- Security issues
- Crash or corrupt the system
- Open space for attack
- Examples
 - ~ Hardcoding credentials
 - ~ Data/SQL injection

```
String accountBalanceQuery =  
    "SELECT accountNumber, balance FROM accounts WHERE account_owner_id = "  
    + request.getParameter("user_id");  
  
try {  
    Statement statement = connection.createStatement();  
    ResultSet rs = statement.executeQuery(accountBalanceQuery);  
    while (rs.next()) {  
        page.addTableRow(rs.getInt("accountNumber"), rs.getFloat("balance"));  
    }  
} catch (SQLException e) { ... }
```

Source: performice.com

```
1.  
2. import pyodbc  
3. info = {  
4.     'user': 'root',  
5.     'password': 'nbusr123'  
6. }  
7. connection_string = "Driver={SQL Server};Server=your_server_name;Database=your_database_name;"  
8. connection = pyodbc.connect(connection_string, attrs_before=info)  
9. connection.close()
```

Issue detection – Code smells

- Maintainability issues
- Decrease readability, architecture quality,...
- Examples

~ Unused private method

~ Feature envy

~ ...

List of code smells:

<https://sourcemaking.com/refactoring/smells>

```
1. class Customer:
2.     def __init__(self, name, address, phone_number, email):
3.         self.name = name
4.         self.address = address
5.         self.phone_number = phone_number
6.         self.email = email
7.
8.     def place_order(self, prod_name, prod_price, quantity, ship_method, pay_method):
9.         ...
10.
11.     def get_invoice(self, order_id, order_date, order_total, pay_status, ship_status):
12.         ...
```

Issue detection – Code smells

- Maintainability issues
- Decrease readability, architecture quality,...
- Examples

~ Unused private method

~ Feature envy

~ ...

List of code smells:

<https://sourcemaking.com/refactoring/smells>

```
1. class Customer:
2.     def __init__(self, name, address, phone_number, email):
3.         self.name = name
4.         self.address = address
5.         self.phone_number = phone_number
6.         self.email = email
7.
8.     def place_order(self, prod_name, prod_price, quantity, ship_method, pay_method):
9.         ...
10.
11.     def get_invoice(self, order_id, order_date, order_total, pay_status, ship_status):
12.         ...
```

Primitive obsession

Issue detection – Code smells

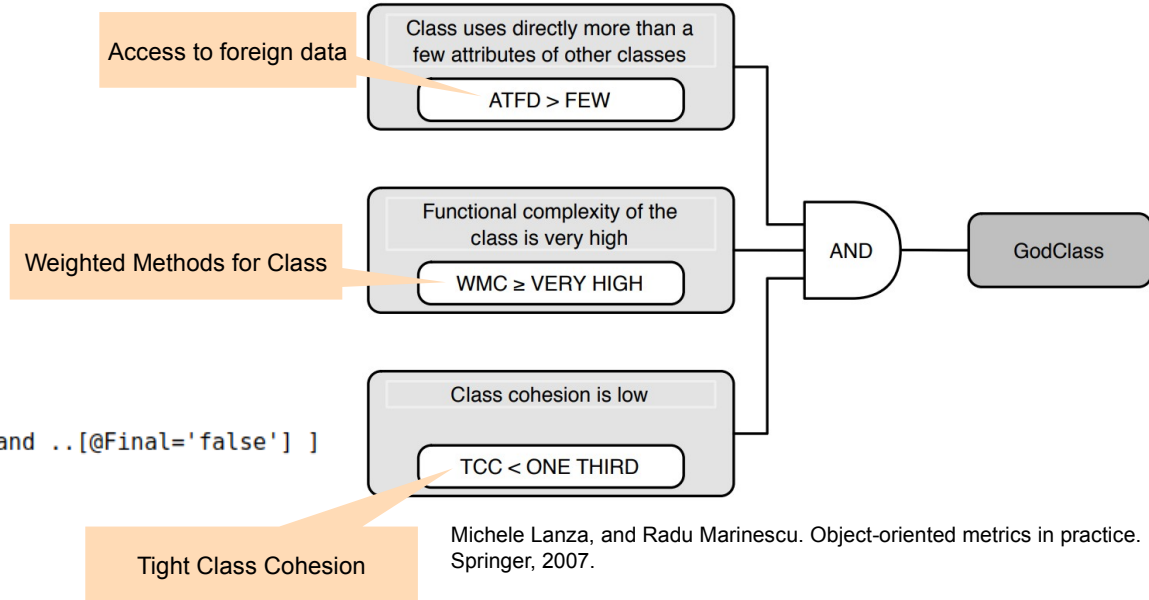
- Pattern-based

```
public class a {  
    Factory f1;  
  
    void myMethod() {  
        Factory f2;  
        int a;  
    }  
}
```

- Factory declaration should be final:

```
//VariableDeclarator[.. /Type/Name[@Image='Factory'] and ..[@Final='false'] ]
```

- Metric-based



Michele Lanza, and Radu Marinescu. Object-oriented metrics in practice. Springer, 2007.

Tools

- Linters
 - PyLint, Prospector, Checkstyle, Resharper, Findbugs, PMD, Sonarlint,...
- Metrics analyzers
 - SourceMeter, Jhawk, Codecity plugins, Pharo, ...
- Code quality analysis platforms
 - SonarQube, Klocwork, Coverity, ...



Sonar Products



- Code quality analysis platform
- Reliability, security, maintainability issues
- Test coverage
- Code duplications
- Technical debt
- Trends
- Multi-language support
- CI integration
- Plugin support
- Self-hosted
- Free community version



- Most features of SonarQube
- Cloud-based
- No plugins
- No free versions



- Real-time issue detection
- IDE integration
- Free

SonarQube - Task



Browse through projects In the SonarCloud demo server:

- <https://sonarcloud.io/explore/projects>
- Focus on projects with higher number of issues
- Explore the issues
 - Try to look for something interesting, e.g.
 - What are the main common types of issues?
 - Where are they located?
 - What kind of functionality is usually affected, etc.?

Week	Lecture	Topic	Assignment deadlines (Tuesdays 10:00 unless otherwise specified)
36	3.9.2024	Introduction and practicalities	
37	10.9.2024	Software quality	Individual assignments 1
38	17.9.2024	Software testing: levels, test case analysis and design	Group registration (DL: 20.9.)
39	24.9.2024	Testing techniques: Black-box, white-box testing	Individual assignments 2, Group assignment 1
40	1.10.2024	Testing techniques: Manual testing	Individual assignments 3
41	8.10.2024	Testing techniques: Black-box, white-box, manual testing	Individual assignments 4
42	15.10.2024	(No lecture)	
43	22.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 5
44	29.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 6, Group assignment 2
45	5.11.2024	Guest lecture	Individual assignments 7
46	12.11.2024	Continuous Integration and Continuous Delivery/Deployment, Test management, and Course summary	Individual assignments 8
47	19.11.2024	(No lecture)	Individual assignments 9
48	26.11.2024	(No lecture)	Individual assignments 10, Group assignment 3

Next steps

- Individual assignments in MyCourses
 - Acceptance testing 2.2
 - Static code analysis 1
- Group assignments in MyCourses
 - Team testing

Deadline: before next lecture (Tuesday by 10:00)

- Individual assignment in MyCourses
 - Static code analysis 2

Deadline: before the next lecture in two weeks (5.11., by 10:00)

- Getting help
 - Ask in the course chat, see Communication section in MyCourses
 - Personal questions by email

Use the related reading
for more details.
Reserve enough time to
set up your environment.



Study smarter, not harder!

- Plan: when and where
- Go deep at your own pace
- Get some rest
- Practice makes perfect
- Enjoy it ◀◀